ALU Class Hierarchy and CPU Integration

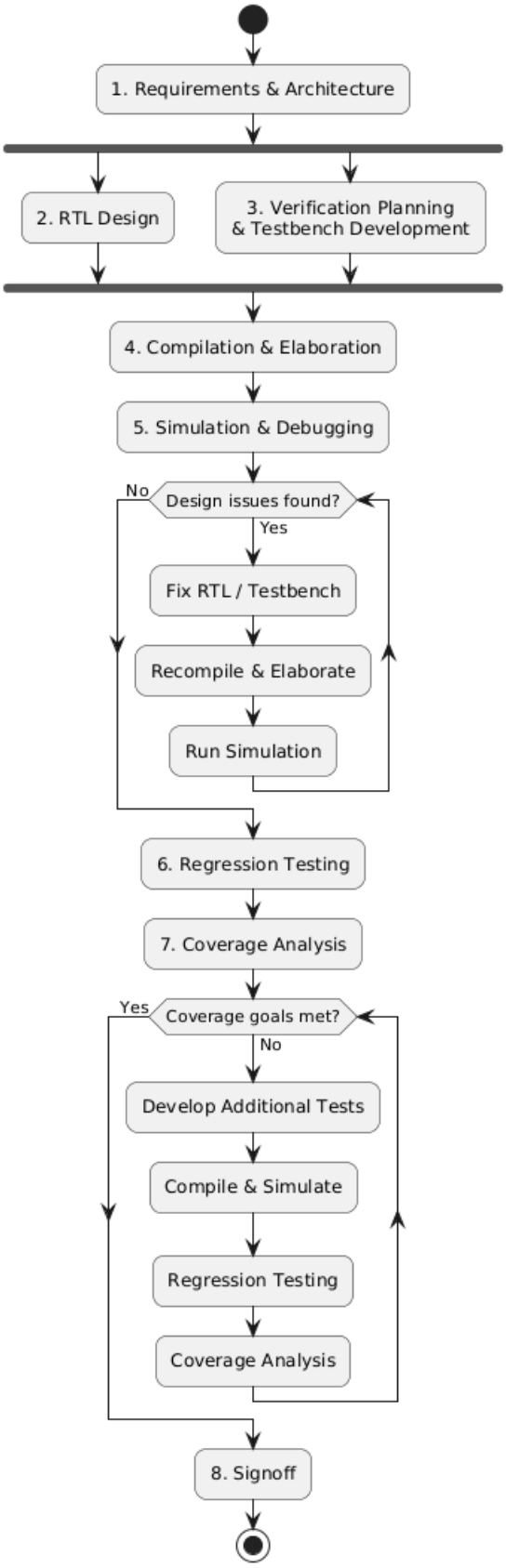
Project Phase Documentation

Description, Inputs and Outputs

Repository: github.com/eslamfawzy72/alu

October 2026

Full Flow


1. Requirements & Architecture
2. RTL Design
3. Verification Planning & Testbench Development
4. Compilation & Elaboration
5. Simulation & Debugging
Design issues found?
No
Yes
Fix RTL / Testbench
Recompile & Elaborate
Run Simulation
6. Regression Testing
7. Coverage Analysis
Coverage goals met?
Yes
No
Develop Additional Tests
Compile & Simulate
Regression Testing
Coverage Analysis
8. Signoff

Project Overview

- The project builds and verifies a SystemVerilog class model of an Arithmetic Logic Unit.
- A common base_alu class holds the operands, operation code and result; logical_alu (AND, OR, XOR, shifts, rotations) and arithmetic_alu (select, increment, decrement, add, subtract, with carry and borrow) extend it; and a CPU class contains one of each and exposes Reset and Calc. All classes live in the package cpu_pkg, each in its own file.
- The work is split into eight phases. Each phase reads the earlier phases' folders directly and never keeps a copy of their files.
- When a phase needs something, an earlier phase has not provided, it writes the question to a questions_for_phase1.md file in its own folder, and Phase 1 answers by updating its files under the same IDs.
- Fixes always happen in the phase that owns the file: design errors in Phase 2, test-program errors in Phase 3, spec gaps in Phase 1.

Phase summary

Phase	Name	Main inputs	Main output
1	Requirement Gathering	Assignment specification	Requirements, decisions, golden vectors, alu_spec.json
2	RTL Design	Phase 1 spec and decisions	The cpu_pkg class model, tagged release phase2-rtl-v1
3	Verification	Phase 1 tests and vectors, Phase 2 design	Verification plan, test program tb_cpu.sv, results
4	Compilation & Elaboration	Phase 2 design, Phase 3 test program	Clean simulation binary and elab_report.md
5	Simulation & Debugging	Phase 4 build, Phase 3 regression	Simulation runs and defects.md
6	Regression Testing	Phases 1–5 scripts and files	Per-run evidence folders, history.csv, report
7	Coverage Analysis & Closure	Phase 6 run, Phase 3 map, Phase 2 RTL	Coverage results, coverage_report.md, exclusions
8	Verification Signoff	Evidence from all phases	signoff.md, signoff_check.txt, signoff tag

Tools used across the project: Verilator 5.020 or later on Ubuntu (WSL on Windows), bash scripts, and python3 for traceability and signoff checks.

Phase 1 — Requirement Gathering

Description

Phase 1 analyses the assignment specification and turns it into structured, traceable records so that the design and verification phases know exactly what the system must do before any code is written. It captures the project overview, functional requirements, interfaces and signals, operations, constraints and assumptions, verification requirements, open questions, traceability and a high-level architecture.

Every open question is answered by a design decision (D-001 to D-018). A decision only becomes a requirement once it is approved. The current output is **revision 3** (spec_version 1.2), in which all 18 decisions are approved.

Phase 1 is also the single source of truth for later phases: when Phase 2 or Phase 3 finds a gap, it writes the question to a questions_for_phase1.md file in its own folder, and Phase 1 answers by updating its files under the same IDs and raising spec_version. Later phases read the files in place and never copy them.

Inputs

Input	Used for
Assignment specification (PDF or Markdown)	Source of truth for every extracted requirement, interface and operation
questions_for_phase1.md from Phase 2 and Phase 3	Gaps found by later phases, answered by updating the Phase 1 files

Outputs

File	Content	IDs
ALU_Project_Overview_Mapping.md	Project overview: name, purpose, components, scope	—
ALU_Functional_Requirements.csv	Functional requirements	FR-001 to FR-013
ALU_Interfaces_and_Signals.csv	Classes, properties, methods and signals	IF-001 to IF-016
ALU_Operation_Behavior_Table.csv	Behaviour of every opcode	OP-001 to OP-014
ALU_Constraints_and_Assumptions.csv	Constraints and assumptions, with approval status	CA-001 to CA-018
ALU_Verification_Requirements.csv	Candidate tests and pass criteria	TC-001 to TC-028
ALU_Open_Questions.csv	Missing or ambiguous information	Q-001 to Q-018
ALU_Traceability.csv	Links from every item back to the specification	TR-001 to TR-108
ALU_Architecture_Corrected.svg	High-level architecture diagram	—
ALU_Design_Decisions.csv	One answer per open question, with approver and date	D-001 to D-018
ALU_Golden_Vectors.csv	Exact expected results incl. boundary cases	Vectors 1 to 37
alu_spec.json	Machine-readable spec that later phases read first (types, classes, operations, golden vectors, tool gate)	spec_version
REVISION_2.md, REVISION_3.md	What changed between revisions and why	—

Phase 2 — RTL Design

Description

Phase 2 implements the ALU as a SystemVerilog class model in the package cpu_pkg: a base_alu class, the derived logical_alu and arithmetic_alu classes, and a CPU class that contains one of each and provides Reset and Calc.

The work is a manual, step-by-step flow through eight numbered folders, from checking the spec to releasing a tagged design.

Its rules keep the design tied to Phase 1: names, types and signatures come from alu_spec.json and the interfaces table and are never renamed; any behaviour the assignment does not define must come from a decision (D-xxx), otherwise Phase 2 asks rather than invents; anything added beyond the spec is recorded in the design plan; and if a lint error shows the plan was wrong, the plan is changed before the code.

Each line that implements a spec ID carries a // @ID comment.

The current release is tag phase2-rtl-v1: lint with Verilator -Wall gives 0 messages and the smoke test reports SUMMARY pass=91 fail=0.

Inputs

Input	Used for
Phase 1 folder (read in place, read-only)	All 61 requirement IDs to implement
alu_spec.json, ALU_Interfaces_and_Signals.csv	Exact names, types and method signatures
ALU_Design_Decisions.csv	Behaviour the assignment leaves undefined (reset state, invalid codes, carry, shifts)
ALU_Golden_Vectors.csv	Expected values for the smoke test
Defect reports from Phases 4 and 5	DUT errors to fix and re-release under a new tag

Outputs

Folder	Content
01_spec_check/	s1_intake_log.md (what was received, counts, inconsistencies) and questions_for_phase1.md
02_requirements/	requirements_map.csv: what each ID means in code and which file owns it
03_architecture/	design_plan.md: files, members, datapath, control, reset, added elements, deviations
04_rtl/	The design: cpu_pkg.sv, base_alu.svh, logical_alu.svh, arithmetic_alu.svh, CPU.svh, and the build list files.f
05_build_checks/	run_checks.sh, lint.log (must be empty), checks_log.md
06_repairs/	repair_log.md: every error, its fix, and whether the plan changed
07_smoke/	run_smoke.sh, smoke_tb.sv, smoke_log.txt, smoke_result.md (golden vectors, reset, invalid codes)
08_release/	release_checklist.md, phase2_handoff.md (tag, build list, compile line, notes for testing) and the Git tag

Phase 3 — Verification

Description

Phase 3 checks that the Phase 2 design does what Phase 1 specifies. It delivers a verification plan, the self-checking test program required by FR-013, and the regression results.

The rule is that Phase 1 decides what is correct and Phase 3 decides how to test it. Every expected value and pass criterion must cite a Phase 1 ID (TC-, OP-, D- or a golden vector). Before testing, Phase 3 runs a ten-point readiness check (R1 to R10) on its inputs; any failed row becomes a question for Phase 1 (or a report to Phase 2) instead of being filled in by Phase 3.

The test program reads the golden vectors from the Phase 1 CSV at run time, has one task per TC, and adds a random test against a reference model plus 44 coverage bins. An expected value is never edited to make a test pass.

Inputs

From	File	Used for
Phase 1	ALU_Verification_Requirements.csv	The 28 tests and their pass criteria
Phase 1	ALU_Golden_Vectors.csv	Exact expected values, read by the program at run time
Phase 1	alu_spec.json	API, enum and operation semantics for the reference model
Phase 1	ALU_Design_Decisions.csv	D-001 to D-018: state, invalid codes, display format, test program content
Phase 1	ALU_Open_Questions.csv	Must contain no open question
Phase 2	08_release/phase2_handoff.md	DUT files, filelist 04_rtl/files.f, and DUT behaviour the tests must allow for (flags F1, F2)

Outputs

File	Content
verification_plan.md	Inputs used, readiness result, stimulus, checking, coverage, exit criteria, status
test_map.csv	Where each TC, OP, D and IF ID is checked, how, its coverage bins, and the reason for anything not tested
tb/tb_cpu.sv	The test program: golden vectors, one test per TC, random test with reference model, coverage
run.sh	Builds and runs the test program against the DUT, checks the log, writes results/
results/	summary.txt, sim.log, build.log, run_info.txt, and checker_selftest/
questions_for_phase1.md	Questions raised in the last readiness check
repair_log.md	Every fix to the test program or run.sh that a later phase asked for

Phase 4 — Compilation & Elaboration

Description

Phase 4 builds the Phase 2 design and the Phase 3 test program into one simulation binary with Verilator. It proves that the two phases fit together before Phase 5 runs anything: both are linted with -Wall and then built with warnings treated as errors.

Phase 4 fixes nothing itself. A testbench error is fixed in Phase 3, a design error is fixed in Phase 2 and released under a new tag, and each warning is either fixed at its source or waived with a written reason. The phase is done when there are zero errors, every warning is dispositioned, and the binary builds from a clean checkout with one command (bash build.sh). Current result: all 31 warnings fixed, none waived.

Inputs

From	File	Used for
Phase 1	alu_spec.json → package.includes_in_order, tool_gate	Include order to confirm and tool-gate checks to tick
Phase 2	08_release/phase2_handoff.md	Release tag, filelist 04_rtl/files.f, compile line
Phase 2	04_rtl/	The design under test
Phase 3	tb/tb_cpu.sv	The test program (top module tb_cpu)

Outputs

File	Content
build.sh	Lints design + test program with -Wall, then builds build/obj/tb_cpu with warnings fatal
Simulation binary build/obj/tb_cpu	The executable Phase 5 runs (in build/, ignored by Git)
build_info.txt	Date, commit, Verilator version, host, top, sha256 of every input, elaborated classes
lint.log, build.log	Tool output of the lint and build steps
elab_report.md	Tool, top, files, include order, every warning and its disposition, tool-gate checks, result
waivers.vlt	Verilator waivers, one per waived warning (only created if needed — none so far)

Phase 5 — Simulation & Debugging

Description

Phase 5 runs the Phase 3 regression on the binary that Phase 4 showed builds. It runs the checker self-test, seed 7, and seed 1 twice, checks that results are reproducible and that a different seed really changes the stimulus, then triages every failure to its owner and tracks each defect until it is closed.

A failing check is classified as a design bug (fixed by Phase 2), a test-program or reference-model bug (fixed by Phase 3), or a spec gap (answered by Phase 1). It is written into defects.md before anything is fixed, and expected values are never edited. Debugging is done at a fixed seed with $display traces, because Verilator does not trace class members in waveforms.

Current status: checker self-test OK, seed 1 and seed 7 PASS with 44/44 coverage bins, seed 1 reproducible, no defects found.

Inputs

From	File	Used for
Phase 4	elab_report.md	Must show zero errors and every warning dispositioned
Phase 3	run.sh, README step 6	The regression and the triage rules
Phase 2	08_release/phase2_handoff.md	DUT tag; flags F1 ($error stops Verilator) and F2 (carry is state)

Outputs

File	Content
simulate.sh	Runs the self-test, seed 7 and seed 1 twice through the Phase 3 run.sh; checks reproducibility
runs/<name>/	Per run: summary.txt, sim.log, build.log, run_info.txt (selftest, seed7, seed1_a, seed1_b)
runs/summary.txt	One line per check plus the overall verdict
defects.md	One row per defect, from first FAIL to closure
Updates to Phase 3	../phase 3/results/ and section 9 of verification_plan.md

Phase 6 — Regression Testing

Description

Phase 6 is the repeatable gate between a change and the next phase. After any change it reruns every existing check — build gate, smoke test, checker self-test and the regression at many seeds — and keeps complete evidence of each run: what was tested (commit, RTL tag, spec version, tool, sha256 of every input), the logs, a pass/fail summary, a per-TC report and a comparison with the previous run.

It runs at three levels: quick (seed 1, about 1 minute), full (seeds 1 to 20 at RANDOM_N=5000, about 6 minutes) and release (full plus seed 99 at RANDOM_N=100000). A failing seed is rerun once; a different result is marked FLAKY, which also fails the run because the design is deterministic for a given seed.

Phase 6 fixes nothing outside its own folder: failures are triaged by Phase 5 and fixed in Phase 2 or Phase 3. The run of record (commit 2deaa9f) passed with 20/20 seeds, 28/28 TCs and 44/44 bins in every seed.

Inputs

From	File	Used for
Phase 1	ALU_Golden_Vectors.csv, alu_spec.json	Expected values (read by the test program); spec_version recorded per run
Phase 2	04_rtl/	The design; RTL tag phase2-rtl-v* recorded per run
Phase 2	07_smoke/run_smoke.sh	Smoke gate
Phase 3	run.sh, tb/tb_cpu.sv	The regression, one call per seed
Phase 4	build.sh	Build gate: lint -Wall and build, warnings fatal
Phase 5	defects.md, README step 2	Where failures are triaged and tracked

Outputs

File	Content
regress.sh	The regression script; writes everything below
results/<date>_<commit>_<level>/	One folder per run, never edited afterwards: run_config.txt, regression_summary.txt, steps.txt, test_report.txt, comparison.txt, failures.txt (on FAIL), and build/, smoke/, selftest/, seed<N>/, long/ logs
history.csv	One appended row per run: commit, RTL tag, spec version, level, seeds pass/fail/flaky, verdict, note
regression_report.md	The runs of record, why each was run, what the comparison shows, and the status of every failure

Phase 7 — Coverage Analysis & Closure

Description

Phase 7 measures how complete the verification is against four goals in coverage_goals.md: G1 functional coverage (all 44 bins hit in every seed), G2 tests (28/28 TCs pass, none partial), G3 requirements (every FR, IF, OP and D covered or excluded) and G4 code coverage (every RTL point hit, every unmeasured line covered or excluded).

Each gap is closed in order of preference: test it (new or stronger check in Phase 3), map it (a passing test already checks it but test_map.csv did not say so), or exclude it with written evidence that a reviewer approves. A gap is never closed by removing a bin, loosening a check, editing an expected value or lowering a goal.

Current status: G1, G2 and the G4 RTL points (35/35) are met; G3 and the unmeasured G4 lines wait on 7 proposed exclusions (D-011, D-013, D-014, D-016, RTL-1 to RTL-3) that have not been approved yet.

Inputs

From	File	Used for
Phase 6	results/<run>/seed*/sim.log, test_report.txt, run_config.txt	Per-bin hits, TC results, what was tested
Phase 3	verification_plan.md §7, test_map.csv	The 44 bins; which TC checks each OP, D and IF
Phase 3	tb/tb_cpu.sv	Prints COVERAGE BIN <name> <hits> per bin
Phase 2	04_rtl/rtl/, 02_requirements/requirements_map.csv	Code coverage; // @ID tags and owner file per ID
Phase 1	Functional requirements, interfaces, operations, decisions, verification requirements CSVs	Every ID to trace; TC → FR links

Outputs

File	Content
coverage_goals.md	Goals G1 to G4, status words, closure rules
merge_coverage.sh	G1: merges per-bin hit counts of one Phase 6 run over all seeds
trace_requirements.py	G2, G3: status of every FR, IF, OP, D and TC in one Phase 6 run
code_coverage.sh	G4: RTL line/block/branch coverage with verilator --coverage
results/<phase 6 run>/	coverage_merged.txt, requirement_coverage.csv, requirement_summary.txt
results/code_<date>_<commit>/	summary.txt, rtl_points.txt, annotated/ sources, coverage.dat, logs
coverage_report.md	Each goal against its measured result, the gaps found, how each was closed, what is left
exclusions.md	One row per proposed or approved exclusion, with reason, evidence, reviewer and date

Phase 8 — Verification Signoff

Description

Phase 8 checks every release criterion against evidence on one frozen commit, records the accepted RTL revision and the final verification status, and — once the approvers agree — tags that commit alu-signoff-v1.

Signoff is given on evidence only: every checklist item links to a file produced by a real run or to an approval with a name and a date. Anything that depends on an unapproved decision, an open question or an unapproved exclusion is provisional, and any change to an input after the release run means starting again from Phase 4. A script, signoff_check.py, checks the six-item checklist mechanically and reports READY or NOT READY.

Current status: **NOT SIGNED OFF** (6 FAIL, 2 MANUAL). The design passes every regression and no defect is open; the blockers are unanswered Phase 2/3 questions to Phase 1 (including deviation DV-2), the 7 unapproved exclusions, and the release regression that has not been run yet.

Inputs

From	File	Used for
Phase 1	alu_spec.json, ALU_Design_Decisions.csv, ALU_Open_Questions.csv	Final spec_version, approved decisions, no open question
Phase 2	Tag phase2-rtl-v*, 01_spec_check/questions_for_phase1.md, 08_release/	Accepted RTL revision; deviations DV-1 and DV-2 accepted
Phase 3	questions_for_phase1.md, repair_log.md	No open question; test-program history
Phase 4	elab_report.md	Zero errors, every warning dispositioned
Phase 5	defects.md	Every defect closed or formally deferred
Phase 6	history.csv, results/<release run>/, regression_report.md	Release regression PASS on the signoff commit, no FLAKY, every failure explained
Phase 7	results/, coverage_report.md, exclusions.md	Goals G1 to G4 met, exclusions approved
Repo root	AGENTS.md → Known issues	All resolved

Outputs

File	Content
signoff_check.py	Checks the evidence mechanically and writes signoff_check.txt
signoff_check.txt	One PASS, FAIL or MANUAL line per checklist item, with evidence, and the verdict READY / NOT READY
open_issues.md	Every unresolved issue and waiver, with owner, what closes it and whether it blocks signoff
signoff.md	The signoff report: accepted RTL revision, tools, spec version, final regression and coverage, traceability, open issues, final status, approvers
Git tag alu-signoff-v1	Placed on the signoff commit and pushed, by a person or with their explicit go-ahead